天猫新品营销技术团队AI编码实战指南（上）

原创 天猫新品营销技术 大淘宝技术 2026年5月6日 02:42 浙江 578人

本文是关于 AI 辅助编码的全面实战指南，基于天猫新品团队的实践经验，从问题本质到解决方案，从理论框架到实战案例，系统性地介绍如何让 AI 更好地完成大部分需求。

本文分上下两篇，上篇包含：

1. 现状与问题诊断 – 深入剖析 AI 生码的四大痛点（写不对、写不好、写不了、改不动），并从项目知识、用户输入、任务复杂度、自检机制、模型能力等五个维度提供针对性解法。
2. 方法论与优化思路 – 提出"最大化复用、自然语言第一、二八定律"三大核心思想，并沿着"前置准备→开发前→开发中→完成后"的全流程，给出每个节点的可落地优化手段。
3. 分场景实战案例 – 根据验收标准和代码质量要求，将需求分为"需求驱动型"和"工程主导型"两类，通过小二端列表页和C端复杂业务的完整案例，展示不同场景下的最佳实践。

下篇包含：

4. 团队建设经验 – 分享新品团队在小二端（后端全栈化）和C端（视图分离、知识库建设、工作流沉淀）两个方向的探索，包括工具建设、文档沉淀、知识库方案等具体落地内容。
5. 实用技巧集锦 – 涵盖 UI 重构、复杂 Prompt 构建、数据转换、多方案选优、文档生成等常见应用场景，以及严厉语气、合理质疑等提升准确度的技巧。

当前AI生码的主要问题

写不对： AI没有完全按照用户意图完成功能，轻则存在缺陷，重则无法运行	**写不好：** AI产出的代码不符合要求，包括但不限于代码质量/代码风格/实现方案

写不了

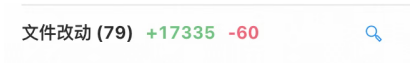
项目隐含逻辑太多，文件结构复杂，耦合度高，AI完全无法按预期完成任务

（如一些内部SDK工具库，使用说明都在外部文档，AI 无法直接通过代码理解如何使用，自然无法写出对应的使用代码）

改不动

在某些迭代场景中，AI一直无法输出正确结果，在错误中不断循环，甚至还可能改坏其他部分，此时只能人工介入

而介入后，想要完成修改则需要面对AI短时间内生成的大量代码，反而导致效率下降，使用者完全丧失了对项目的把控



导致问题的主要因素与解法

1. 项目/需求 隐含信息过多（AI不知道）

由于大家都是淘内私有项目，代码中不仅包含了专属的业务逻辑，还有来自四面八方的SDK工具库代码，面对无处获取的项目知识，即使是Cluade40来了也无济于事。

解法：

- 使用有明确声明的NPM包，或者给其接入Artifact7 等辅助文档生成工具；
- 提供可访问的外部知识库，包括但不限于 MCP工具 / 项目知识库 / 需求文档。

2. 用户输入不精准，必要信息不足（没给AI说）

代码开发就是从模糊的需求转向无歧义的代码，模糊部分在实现中必然会被补充，实现不合预期的一大原因就是模糊部分没有明确说明。

解法：

- 用户主动增加输入内容 / 辅助工具提升输入质量
  - 对一些常见的情况提供 prompt 模版，使用时根据实际需求作部分修改；
  - 借用工具进行用户输入扩写，以及对必要的模糊部分进行标记与阐明；
  - 引入 Spec Coding 方案，以详细的文档作为AI输入；
  - 对高频场景做出约定，通过约定来覆盖模糊（如维护一份持续更新的AGENT.md）。
- 意图识别，基于项目上下文自动推测模糊部分
  - 接入MCP工具，扩大 AI 感知力，使AI能更好地理解用户意图；
  - 通过 代码索引 / 项目文档 / CodeWiki 等辅助手段，使 AI 可以基于项目代码快速仿写与推测。

> 这里存在一个关于文档详细程度的权衡点：到底是使用大而全的文档，还是小而精。
> 经过实践，最适合的方案应该是先给出一个可以基本描述清楚需求的小文档，再根据AI实际产出的偏离情况来进行问题补充，因为大部分都是几个重复出现的常见问题（前提是保持工具和模型尽可能不变），进行几次补充以后就可以完成一份精准好用的输入文档。

3. 任务复杂度高

AI 生码的成功率会随着任务复杂度的提升而在某个节点开始骤降，此时则需要通过合适的手段降低任务的难度，最大化 AI 的生码成功率，而降低复杂度主要有以下两个角度。

- **降低任务复杂度**
  - 识别重复的工作流与代码，进行针对性优化与封装复用，从而减少生码的代码量与不确定性；
  - 复杂任务拆分，将单个不易测试的大型任务，拆成多个可验收的小型黑盒；
  - 固定实现思路/细节，统一思维模型，通过约定来减少 思维/选型 负担。
- **降低工程复杂度（文件量、耦合度）**
  - 借助优秀的工程结构设计，实现 代码 / 模块 / 文件 的天然解耦（很多生码问题其实归类到底都会走到基础的代码工程化问题，具有优秀工程结构的仓库天然就有较高的AI生成成功率）；
  - 通过代码索引/项目文档/CodeWiki 等辅助手段，提高AI检索效率，减少因检索困难而新增的无用上下文。

4. 缺少自检环节

现在的常规生码流程，只会进行基础的代码规范与语法检测，并没有完整的Review & Test 的流程。此时的AI只是完成了代码生成，并不一定完成了任务，也不一定满足实际的代码质量要求。

- **代码质量自检**

- 在本地引入自检流程，及时完成代码质量确认；
- 使用 Code 平台的AI CR助手进行发布前预检，可以自定义 CR 规则。

- **功能自检**
  - 前端可以通过接入MCP的方式让AI可以感知前端页面，从而进行部分功能测试；
  - 后端可以通过单测直接查看功能正确性；
  - 对于无法测试的大型任务，可以拆分到多个易于验收的黑盒（如前端进行视图分离，对逻辑hooks部分进行单元测试）。

5. 模型 / Agent 的差异与能力限制

模型的不同 / 编码工具的不同，都会影响生成结果，即使是同一个模型，也会因为模型的随机性而产生不同的结果。要将 AI 生码运用在工程中，不仅需要面对模型的短板，还需要对抗模型的随机性。

- 上下文窗口有限
  - 留存过程文档，实现上下文复用，跳过收集环节
    - 代码上下文：仓库信息、接口信息
    - 项目上下文：PRD、功能文档
  - 提供更精准的上下文，尽量不让 AI 靠文件猜测
  - 通过一些用户 Rule 监控注意力状态（比如：让 AI 每次对话结束都要用谢谢结尾，如果没有则说明上下文窗口已经爆了）
- 随机性 / 模型差异 导致生成内容的质量不稳定
  - 提供更严格的约束文档 / Spec Coding方案
  - 补充自检环节
- 修改型问题错误率高
  - 修改型问题相当于上下文更多，正确率要求更高的生码任务，而AI理解力弱，所以修改型任务准确率更低，这就是大家最常提到的改不动的问题。对于这个问题，主要的解法就是用前文提到的办法去降低任务复杂度：减少 AI理解代码的难度 或者 降低生码任务体量
- 天生对某些问题能力弱，容易卡在死循环里
  - 识别常见错误场景，By Case 分析积累经验，沉淀对应的解法

核心方法论与全流程优化指南

这部分主要介绍提高AI生码效果的一些核心思路，并基于生码流程中的各个节点给出一些可以实施的优化手段。

核心思想

- 最大化复用

无论是人工编码还是AI编码，最有效率的提效方案就是提高代码的复用度。 在AI编码的背景下，通过复用还可以提高生码任务的确定性，极大降低任务复杂度，提高对代码的掌控度，保证生成结果的质量。

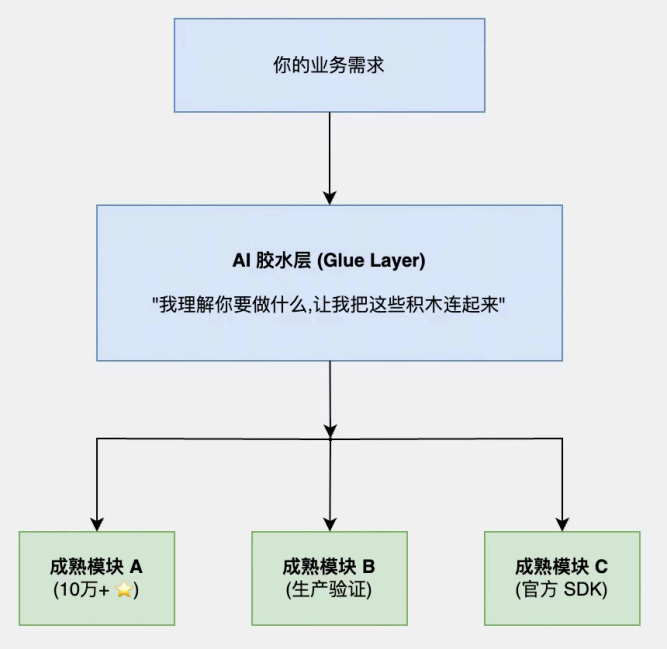
模块优先

将每一部分的开发都视为一个明确输入输出的标准可复用模块（可信任的黑盒），且所有相关声明可以通过明确的路径访问获取，每个功能都是具有清晰边界的库，AI可以很自然地从代码中获取所需知识。

（现在的AI工具已经具备了很强的信息获取功能与推理能力）

胶水编程

优先使用已有模块，不要自己造轮子，通过最小量的“胶水代码”将它们组合成完整系统，你的代码只负责：组合、调用、封装、适配。从而在使用AI完成大部分代码的同时，仍然保留项目掌控度。



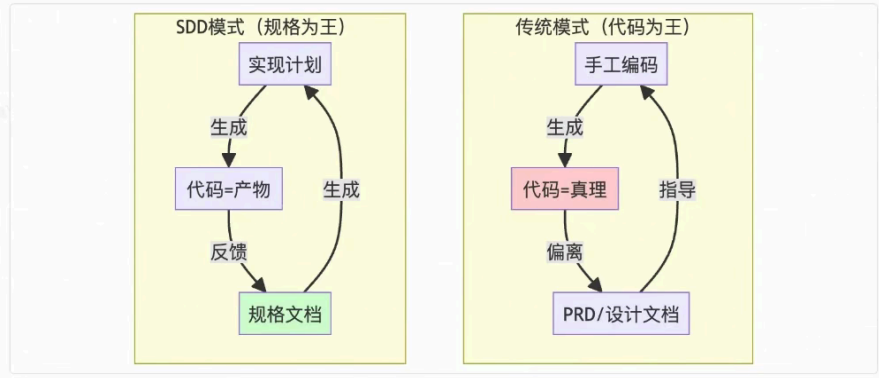
工作流复用

对于一些可标准化/重复性强/复杂度高的工作流程，可以及时沉淀到标准的AI工作流（包括但不限于 文档、MCP、Skill），进一步扩大工作中的可复用范围。

- 文档先行

文档是AI Coding的第一要素，PRD即单测，文档即代码，优先修改文档而不是代码，这部分文档并不要求大而全，重点是对大致方向的描述，与部分易混淆部分的详细说明。

基于 Spec-kit 的模型，理想状态下是有一份可以和代码100%互相转换的文档，但是现在实测下来很难达到这种理想状态（没有可以完美描述项目的文档，文档也没法恰好转换到代码），最好的策略还是够用就行。



- 二八定律

- 认清AI的28 定律，20% 时间可以完成80%的任务，但是剩下的 20% 要 80% 的时间；

- 0% → 80%（蜜月期）：从零开始生成新功能非常快，AI 对全新的、独立的逻辑处理得极其完美；
- 80% → 100%（深水区）：当功能需要收尾，涉及到复杂的上下文、边缘 Case 修复、以及与旧逻辑的耦合时，AI 的表现会断崖式下跌，常常需要人工介入；
- 提高AI生成效率的重点，1是提高前80%的质量，2是提高后20%的效率。

全流程节点

一次 AI Coding 的执行流程大概可以概括为：理解用户意图 > 查找上下文> 设计执行方案> 代码生成 > 简单校验。由于 LLM 的随机性，每一个执行环节都包含着无限的可能，而掌控AI生码，就是在每个环节都进行明确声明与介入，从而保障AI按照自己的预期进行产出。

以下是从每个节点进行拆分，关于AI编程流的每个部分，我们有什么手段可以进行优化。

> 这部分主要是基于每个节点进行优化方案讲解，并不代表实际需要严格按照这个步骤进行流程拆分，按需选用即可，具体实践中的取舍和选型在后面的实战案例中会详细说明。

- 前置准备

- 准备项目/团队知识库，提供公共组件API，业务知识，包括但不限于 mcp / 知识库 / 文档
- 确定基础的代码实现规范，约束代码风格，通常命名 README.md / AGENTS.md 放在根目录，如果AI没有读取则指明（通过rules或者手动添加上下文）
- 以尽可能明确、解耦的形式，设计仓库的目录结构与实现方案
  - 对于文件引用层级多的情况，AI出码的正确率显著下降（一部分是因为前端属于弱类型语言）

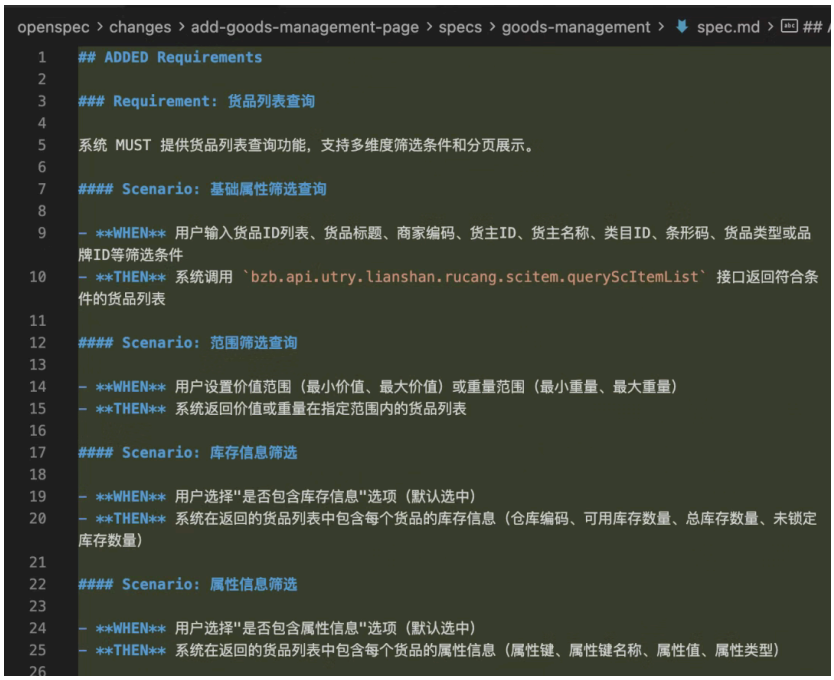
  - 对于更加复杂的项目，可以尝试使用一些更进一步的视图分离与状态管理等方案进行提前解耦。

- 开发前

明确需求内容（代码无关）

产品开发就是从模糊的文档转向代码，而代码是无歧义的，模糊部分在实现中必然会被补充，实现不合预期的一大原因就是模糊部分没有明确说明。

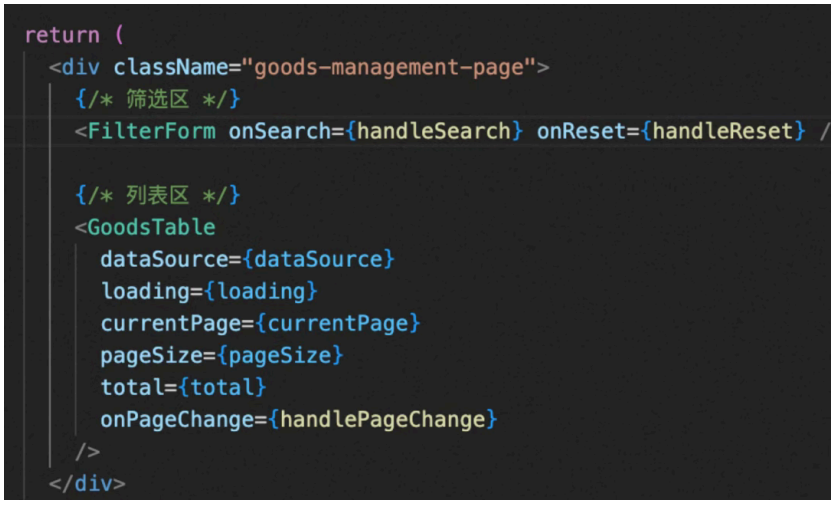
- 对于不明确的情况，可以使用一些辅助工具或者基础模版进行标准化 prd 的产出，并根据需求进行修改；
- 需求明确部分，不建议涉及具体实现，目前的AI编码能力已经足够强大，重点说明功能需求即可，明确的功能文档足以让 AI 设计出合适的方案，提前涉及实现不仅影响需求说明，也会限制 AI 的发挥；
- 如果需要进一步明确各部分功能点，可以通过 spec 工具转换成近似单元测试的功能文档，必要时将其转化成实际单元测试。



任务设计与拆分

如果单次实现过于复杂，会丢失对代码的掌控度；而如果单次任务体量太大，AI 容易在执行过程中丢失上下文与注意力。

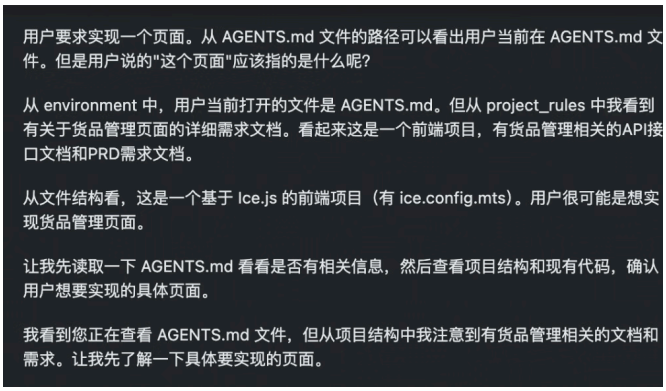
- 组件拆分：将一个复杂、验收卡口严格但无法直接测试、涉及模块多、迭代频率高的大规模任务，拆分到多个简单、迭代频率低、可测试的黑盒，并进行组装。



- 流程拆分：将复杂任务进行分步拆解，并通过清单文档记录完成情况。

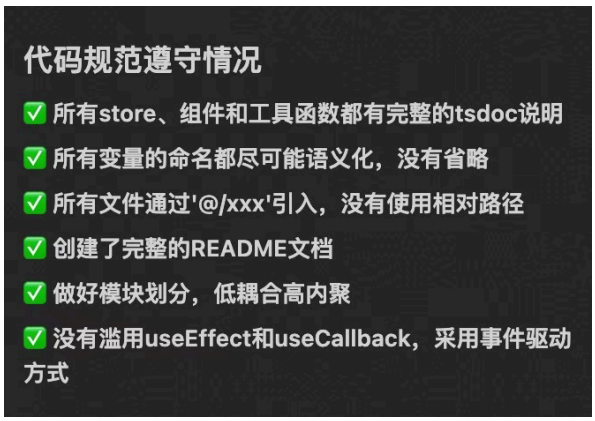
- 开发中（新建型需求）

- 创建并持续迭代过程文档，如页面README，组件说明，从而保障每次对话时AI可以快速获取信息，避免AI读取过多文件。



（AI每次新对话都需要花费大量token在读取与需求无关的前置上下文）

- 及时管理上下文窗口，确保AI没有失去专注度（比如让AI每次执行完重述基础原则）



- 尽可能使用严格解耦的架构设计，防止留下技术债
- 引入自检机制，保障功能与质量
  - 让AI自行添加调试点位，通过调试信息修改问题

    - 对黑盒 组件/功能 创建单元测试
    - 通过MCP工具进行页面测试
    - 使用AI单独进行代码质量Review环节
- 对于部分持续失败的场景
    - 尝试修改用户输入，提供更精确的上下文，或者向AI提供方向指引
    - 让AI在修改代码前先输出方案并审阅
    - 收集到案例中进行By Case分析

常见陷阱：在一次会话中进行大量任务，这会导致上下文太长太宽泛，模型注意力丢失。

- 对于独立的任务，应该及时新建对话
- 对于大型任务，应该及时生成各类过程文档，当模型成功率显著降低时切换对话并传入过程文档恢复上下文

- 开发中（迭代型需求）

常见问题：

场景	发生了什么	后果
改一个功能，坏了另一个	添加删除功能时，不小心影响了添加功能	花时间排查，可能越改越乱
想回到"昨天那个版本"	昨天的代码能用，今天改了一堆，全坏了	找不到昨天的版本
试了三种方案，想回到第一种	第一种方案其实最好，但已经被覆盖了	要么重写，要么将就
AI 改了不该改的地方	让 AI 改一个文件，它顺手改了其他文件	不知道哪些被改了

相较于新建型任务，迭代时成功率更低的原因主要是：AI天生重模仿而弱理解，新建型任务侧重于模型对代码的仿写能力，而迭代型任务要求AI理解代码且精准修改，所以成功率更低。

- 注意及时约束，防止AI目标漂移

Bad Case ❌	**Good Case ✅**
"帮我优化这个函数"（结果 AI 重构了整个类）	只优化函数 calculateTotal，不做任何其他的变更，集中于此函数一点

- 尽量传入精准的上下文，减少AI搜索范围

Bad Case ❌	**Good Case ✅**

帮我修改现在的弹窗样式，和其他页面的统一	帮我修改pages/GoodsManagement/drawer.tsx 下的弹窗样式，参考pages/Warehouse/GoodsDetAll/drawer.tsx 目前主流工具都可以添加上下文，不用自己写path

- 对于复杂迭代，最好及时通过git进行版本管理，因为符合需求的代码可能在前几个小时，甚至前一天的某个版本（多方案对比用分支 / 版本管理用commit）

```
todo-list/                    ← 当前开发的版本
todo-list-v1-基础功能完成/     ← 第一个能用的版本
todo-list-v2-添加删除功能/     ← 添加了删除功能
todo-list-v3-数据保存/         ← 添加了 localStorage
todo-list-v4-深色模式尝试/     ← 准备大改之前的备份
```

- 进一步的迭代成功率，主要依赖于仓库架构的解耦程度，改不动时就要依靠极致性的解耦
- 人工必须介入时，可以采用人工提供方案，AI执行改动的半自动方案节省时间（如：我要移除这个函数中硬编码的业务逻辑，将其改成外部传参，并在改好后同步修改所有调用了这个函数的代码）

```
export const makeProductItems = (cartItems = [] as CartItem[]) => {
  return cartItems.map((item) => {
    return {
      itemId: item?.itemId,
      title: item?.tryItemTitle,
      img: item?.tryItemImg,
      isSelected: true,
      selected: true,
      groupName: 'u先vip',
      skuId: item?.skuId,
      panoIdx: 0,

      ...(item?.fpChannel && { fpChannel: item?.fpChannel }),
      ...(item?.fpChannelSig && { fpChannelSig: item?.fpChannelSig }),
    };
  });
};
```

- 实在无法继续迭代时，借助已有的功能文档进行整体代码重构（如果没有，可以尝试让AI根据目前进行总结，但是此时最好从模块向上重构）
- 其余部分同新建型需求

- 完成后

基于需求完成情况及时进行问题点分析与资产沉淀，理想情况甚至可以考虑让 AI 基于已有经验进行自我迭代，从而逐步扩大能力边界，踩过的坑就不要再踩，写过的代码就不要写第二遍。

- 基于生码过程中的常见问题，及时迭代基础规范文档

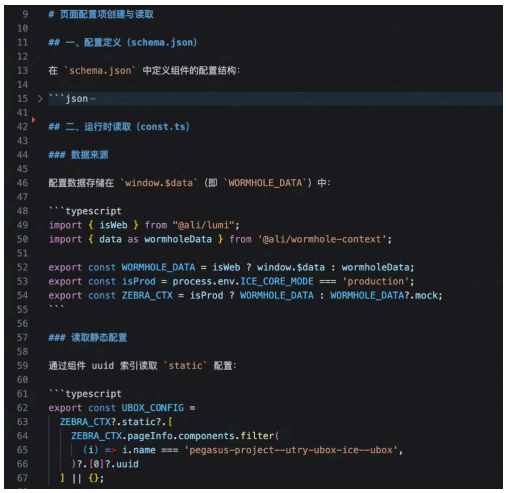
重点注意

仅在必要时使用useEffect，仅在必要时使用useEffect，不要使用useCallback 仅在必要时使用useEffect，仅在必要时使用useEffect

1. 所有组件和工具函数都要有完整的tsdoc说明
2. 所有变量的命名都要尽可能语义化，不要省略
3. 所有文件通过'@/xxx'引入，不要使用相对路径
4. 对于逻辑复杂的部分需要及时进行组件化，不要在主组件中进行复杂的逻辑处理与展示
5. 组件尽量内聚，尽可能减少props传递
6. 使用antd组件库进行实现，代码完成编写后将所有import的antd换成@ali/tao-design

（小二端开发中沉淀的关键注意点，尤其是需要治理AI喜欢乱用hooks的毛病）

- 识别关键流程，沉淀到Skill（资产化的AISOP，包含描述、指令、脚本、模版等内容）



```
# 页面配置项创建与读取

## 一、配置定义（schema.json）

在 `schema.json` 中定义组件的配置结构：

```json
...

## 二、运行时读取（const.ts）

### 数据来源

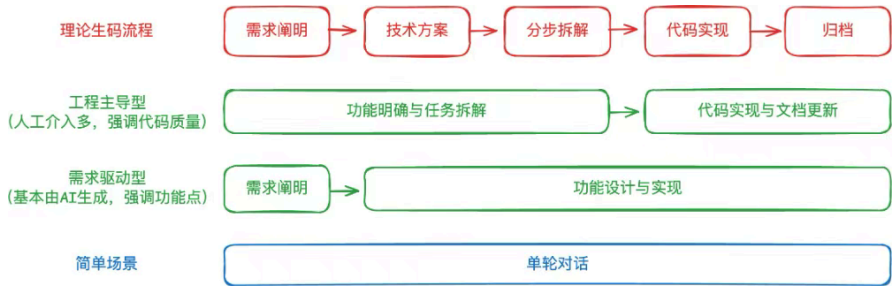
配置数据存储在 `window.$data`（即 `WORMHOLE_DATA`）中：

```typescript
import { isWeb } from "@ali/lumi";
import { data as wormholeData } from '@ali/wormhole-context';

export const WORMHOLE_DATA = isWeb ? window.$data : wormholeData;
export const isProd = process.env.ICE_CORE_MODE === 'production';
export const ZEBRA_CTX = isProd ? WORMHOLE_DATA : WORMHOLE_DATA?.mock;
```

读取静态配置

通过组件 uuid 索引读取 `static` 配置：

```typescript
export const UBOX_CONFIG =
  ZEBRA_CTX?.static?.[
    ZEBRA_CTX.pageInfo.components.filter(
      (i) => i.name === 'pegasus-project--utry-ubox-ice--ubox',
    )?.[0]?.uuid
  ] || {};
```
```

- 识别关键模式，沉淀到标准化 组件 / 模版

| | 出码准确率 | 人工介入成本 |
|---|---|---|
| AI自由发挥 | 70% | 高 |
| 有语料的三方包 | 80% | 低 |
| 私有包+调用规范 | 95% | 低 |

看完以上的内容，有人会说：我只想使用基础的Chat进行代码生成，不想评审AI生成的方案，也不想跳转出去使用额外的工具，怎么让我的Chat生码效果更好？以下是几个快捷可用的调优手段：

- 配置基础的项目规则，并根据实际的问题对其进行一些调优；
- 提前设计较为解耦的项目结构，提高 AI 迭代的成功率；
- 接入一些辅助的MCP工具和已验证过的Skills进行能力辅助；
- 使用一些基础的提示词流程或模版，尽可能描述清楚需求；

当纯对话方式完全陷入瓶颈时，可以参考以下两种实战案例进行优化。

## 03 两类实战案例

AI编码过程中，有个比较重要的关注点就是：在保证迭代成功率的同时，还要留出一定的人为可介入空间，以及保持对代码的掌控度；对于验收要求越高的项目，掌控率和人工可介入空间的要求就越高，而根据验收要求以及实际情况，可以将实际需求分为以下两类。
```



	需求驱动型	工程主导型
特点	强调"要什么功能" AI 自主决策技术实现 人工介入少，关注结果	强调"怎么实现" 人工深度参与实现方案决策 人工介入多，关注代码质量
代码规范度要求	低	高
验收标准	低	高
隐含知识量	低	高
项目复杂度	低	高
AI扮演的角色	功能开发者	编码辅助员
案例	小二后台页面 / 研发自用工具	线上C端页面 / 商家端页面

需求驱动型（DO WHAT）

这一场景的主要关注点在于：

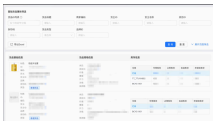
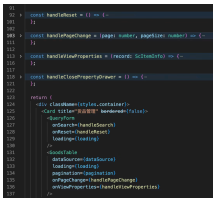
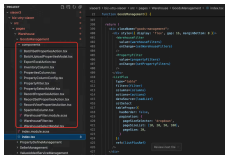
- 想办法完全阐明需求点，防止 AI 臆造或偏离；
- 要有一定的辅助手段来保证代码质量，从而提高迭代成功率；
- 留出一定的人工介入空间，不要产出一堆难以介入的代码；

接下来将以一个小二端的需求，从新建页面到二次迭代功能，逐步对比不同方案产出的生码结果。

- 新建一个小二端列表页

> 需求背景：
> 一个常规的列表页，按字段展示货品相关信息，并且支持按字段过滤。

对于简单的页面生成，提供足够信息的文档，AI即可完成对应的需求，而根据实际需求的验收卡口与用户需求，还可以分为如下三种实现风格。

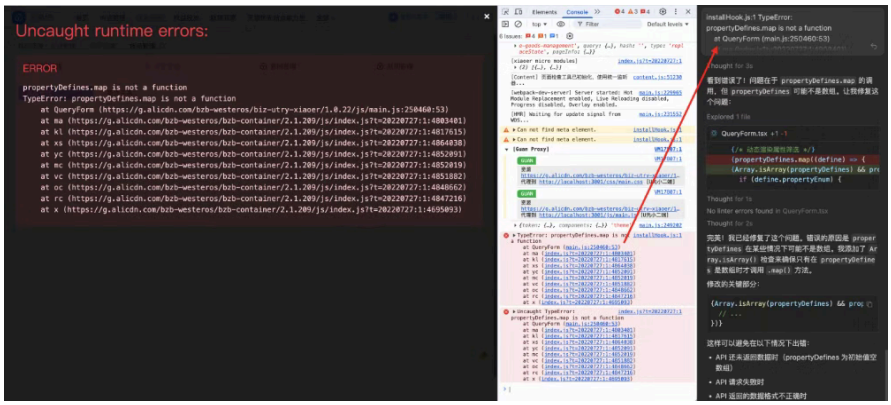
	能用就行型	较有要求型	严格型 （已经有一份关于常规列表查询页的实现模版）
实现路径	 • 直接输入接口文档，其他完全由AI进行完善。 • 提供少量实现限定（使用tao- design），或者由AI自行读取仓库代码进行判断	 • 提供基础的代码实现规范与文件模版 • 按要求提供prd，并进行一定的结构化优化	 • 提供基础的代码实现规范与文件模版 • 按要求提供prd，并进行一定的结构化优化 • 提供更严格的具体实现绑定，通过严格的代码模版限定产物格式
效果			
样式	组件库兜底基本风格	组件库兜底基本风格 功能符合用户要求	符合**视觉稿 / 平台规范**要求 功能符合用户要求
可迭代性	非复杂情况，AI也可以再进行一定程度的迭代	AI可以进行一定程度的迭代	AI可以进行长期多次的迭代
代码质量	 代码组织格式随机（还可能产出单个巨型文件），人工介入成本高	 代码组织格式较规范，代码轻度解耦，人工介入成本中，仍然有部分逻辑代码包含在主文件	 代码完全按照统一思路 / 组件 完成，功能实现分散在各个子组件，代码严格解藕，人工介入成本低 对于这种严格的解耦模式，人工编写时可能成本过高，但是交给AI编写则是适得其所

- 问题调试

分支控制

在微调阶段，及时通过Commit保存版本，因为AI生码是覆盖式，没有及时存档会因为某些错误修改导致代码混乱，前功尽弃（目前的生码工具都有一定的回退功能，但是不能过于信任）。

报错调试



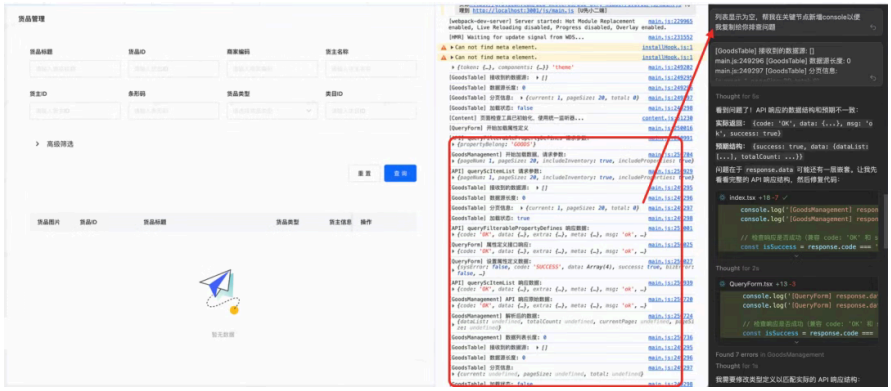
常规报错直接将报错部分复制给Agent自行调试即可，可解决率80%（解决不掉的部分主要来自三方库内部的报错，需要特判）。

部分非阻断型报错，关掉弹窗即可，部分错误弹窗只是便于本地调试，线上并不会显示。

数据调试

数据问题不会有报错提示，可以让AI自行生成一些输出日志辅助排查。

> xxxx显示为空，帮我在关键节点新增console日志，以便我复制给你排查问题。



以上页面调试也可以通过MCP工具，或者直接通过cursor的browser tab进行，篇幅问题不做展开。

- 进行一次涉及多个部分的中型迭代

> 需求背景（原始需求）：

> 在货品管理页面中，货品除了基础信息外，还包含动态属性信息。这些属性由属性定义系统管理，支持多种数据类型（字符串、数字、布尔值、时间戳等），不同货品可能拥有不同的属性。
> 为了提升货品管理的效率和用户体验，需要提供以下功能：
> 属性展示：在列表中直观展示货品属性，支持用户自定义显示哪些属性
> 属性筛选：支持按属性进行筛选查询，快速定位目标货品
> 属性编辑：支持查看和编辑单个货品的属性信息。

明确需求

先基于接口文档和需求，交给AI创建产品文档，核对完成后进入下一步（常规改动可以让AI直接生成，但如果涉及到复杂的交互，最好先看看AI准备怎么实现）,以下是让AI扩写后的产品需求文档。

```
# 货品属性功能需求文档

## 需求背景

在货品管理页面中，货品除了基础信息外，还包含动态属性信息。这些属性由属性定义系统管理，

为了提升货品管理的效率和用户体验，需要提供以下功能：
1. **属性展示**：在列表中直观展示货品属性，支持用户自定义显示哪些属性
2. **属性筛选**：支持按属性进行筛选查询，快速定位目标货品
3. **属性编辑**：支持查看和编辑单个货品的属性信息

---

## 一、功能需求

### 1.1 属性列展示

**功能：** 在货品列表中添加"属性"列，展示货品属性信息。

**交互：**
- 默认显示所有属性，每个属性以"标签+值"展示
- 操作区提供"配置属性列"按钮，可配置显示哪些属性
- 时间戳类型显示为日期时间格式，文本类型支持换行
- 无属性时显示"暂无属性"

---

### 1.2 属性筛选

**功能：** 在筛选区支持按属性进行筛选查询。

**交互：**
```

- 筛选区显示"按属性筛选"区域，已添加的筛选条件以 Tag 展示
- 点击"添加属性筛选"打开弹窗
- 弹窗中选择属性，系统根据属性类型自动判断筛选方式：
  - 字符串/文本：有枚举值→枚举多选，无枚举值→文本匹配
  - 数字：数值范围
  - 布尔：布尔值选择
  - 时间戳：时间范围
  - 时间戳范围：时间点
- 输入筛选值后确定，筛选条件生效并触发查询

1.3 属性编辑

功能： 在操作列提供"属性"按钮，可查看和编辑货品属性。

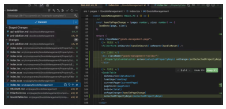
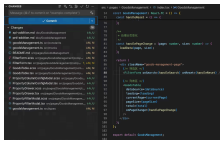
交互：
- 点击"属性"按钮打开右侧抽屉
- 抽屉中以表格展示属性信息（属性键、属性键名称、属性类型、属性值）
- 根据属性类型显示对应编辑控件：
  - 数字→数字输入框
  - 布尔→开关组件
  - 时间戳→日期时间选择器
  - 时间戳范围→日期时间范围选择器
  - 字符串（有枚举值）→下拉多选
  - 字符串（无枚举值）→文本输入框
  - 文本→多行文本输入框
- 修改后点击"保存属性"，保存成功后刷新列表并关闭抽屉

二、数据类型说明

属性类型	筛选方式	编辑控件
STRING	文本匹配 或 枚举多选	文本输入框 或 下拉多选
TEXT	文本匹配	多行文本输入框
NUMBER	数值范围	数字输入框
BOOLEAN	布尔值	开关组件
TIMESTAMP	时间范围	日期时间选择器
TIMESTAMP_*RANGE*	*时间点*	*日期时间范围选择器*

迭代效果对比

由于不是所有页面都能找到可以恰好抽象描述出来的页面模版，所以这里采用 能用就行版 和 较有要求版 进行迭代效果对比。

	能用就行初版 + 仅输入原始需求	较有要求型初版 + 仅输入原始需求	较有要求型初版 + 有人工阐明后的详细功能文档
最终效果	 基本符合要求	 功能做出来了，但是不太合预期	 基本符合要求
代码质量	 大量修改主文件，多次迭代后成功率必然大幅下降 迭代后主文件行数达到600行，基本丧失人工介入可能性	 仅轻微修改主代码 子组件划分清晰，人工介入成本低	 主文件逻辑清晰，迭代未修改主文件 只修改了涉及相关的子组件（子组件其实也可以更内聚，这部分可以让AI再进行二次优化）

由上可见

- 清晰的 prd 保证功能符合预期：如果没有预先对 AI 想要产出的内容进行审核，很容易发生偏离导致结果不合预期；
- 优质的代码提高迭代效率：初始的代码对后续迭代的成功率有较大影响，如果源代码已经在堆砌代码，后续迭代时AI也会延续这个风格，导致迭代成功率急速下降，且丧失人工介入修改代码的空间。

工程主导型（HOW TO DO）

这一场景的主要关注点在于：

- 代码需要严格符合工程质量；
- 编写者要保留对代码大部分的掌控度，且产出内容人工可介入度高；
- 对于这类复杂度高 / 隐含知识多的项目，怎么让 AI 可以做 / 知道做。

- 需求背景与实现拆解

需求内容：

需要在页面Feeds下新增二级类目配置，且商品卡片点击跳转切换到跳转自有中间页（原来是直接跳转商品详情页），完成视觉更新。

实现拆解

服务端	前端
需要新增一个 ald solution，且支持二级类目召回支持（be召回时增加参数）	• 新增二级类目配置项并更改接口参数 • 重写feeds组件（原有旧组件不支持多级 tab），修改卡片与 tab 样式 • 修改卡片跳转逻辑

- 前置知识准备

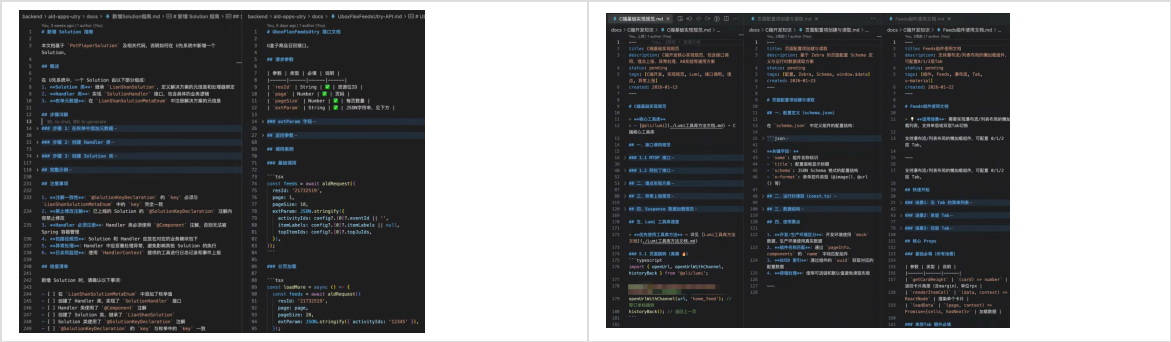
对于这类业务仓库，隐含知识及其多，这些隐含知识有些来自业务语义，有些来自内部平台的开发模式，也有一些来自各自团队的实现规范。如果没有前置输入，AI 完全无从下手，此时需要提前识别业务语义下的隐含知识与一些实现规范 / 方案，并将其沉淀到文档，让 AI 有迹可循。

首先，梳理出当前需求需要声明的隐含知识

- 什么是自建的中间页？什么是商品详情页？怎么进行跳转？
- 后端仓库内怎么新建一个solution，有没有什么相关的代码规范？
- 前端仓库要使用什么工具库？一些基础功能如何实现？
- Feeds组件库如何使用？配置项是什么？怎么更改？

以下是梳理出来的前置文档，文档这部分建议 AI 生成配合人工修改，尽量采用渐进式披露的原则，入口文档信息全而精，对于需要详细介绍的部分，可以另外创建文档进行补充，最好是有一个利于统一读取的地方进行存放（前期冷启动的时候编写前置文档会花较多的时间，但是这是值得，且未来必须要完成的事情，后期复用到的时候就会发现有预制文档有多爽）。

服务端相关文档	前端相关文档

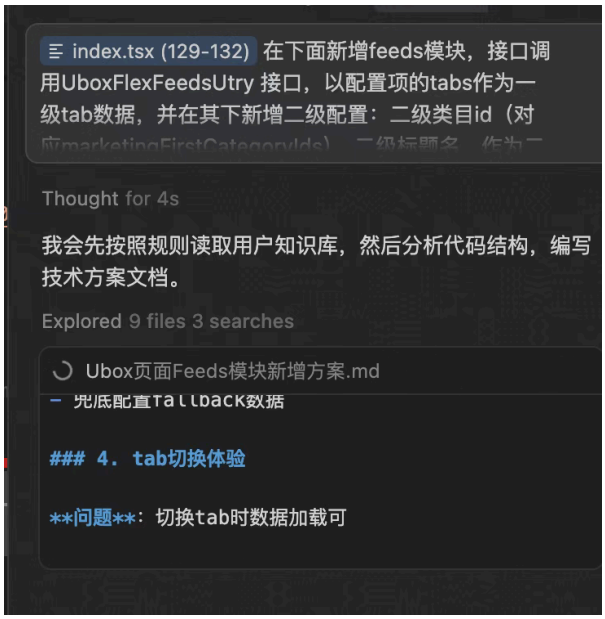


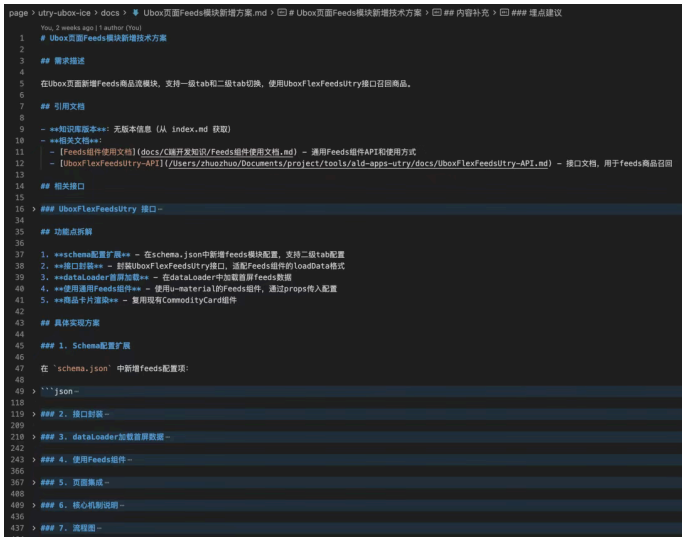
- 技术方案产出

准备好以后就可以基于知识问答和代码产出技术方案，这部分注意还是要提供关键信息，对于 AI 需要从知识库获取知识的情况，最好是让AI列出其参考的具体文档，防止出现偏离。由于是重代码质量的项目，需要人工完成对技术方案的完整审查与修改，也是对代码掌控度的保证（毕竟线上 bug 不能让 AI 背锅）。

以下是采用的技术方案模版与实际产出方案，虽然技术文档看起来行数很多，但是其实大部分都是代码节选部分，限定格式后的技术文档其实不会占用太多的审查时间。

````
实现前首先按照如下模版进行技术方案编写，文档生成到仓库docs目录下，我确认后再进行实现

技术方案模版
引用文档 - 声明引用的知识库文档，并写入从知识库根目录的index.md获取版本号
```
## 引用文档
- **知识库版本**：v1.2.3（从 index.md 获取）
- **相关文档**：
  - [核心业务流程文档](链接) - 用于理解业务逻辑
  - [技术架构文档](链接) - 用于确定技术选型
  - [API 规范文档](链接) - 用于接口设计参考
```

相关接口 - 如无则省略
功能点拆解 - 讲用户的需求分解成具体的功能点
具体实现方案 - 大致声明实现方案，并在关键部位配上核心代码，或者mermAId流程图
问题预警 - 分析整体流程上可能会出现问题的地方，并提供应对措施
相关埋点 - 如无则省略
内容补充 - 可以根据需求内容自由发挥，标题内容自拟，做一些内容补充，但是仅限一小部分
````





可以看到，被前置文档喂饱后，AI很完整地了解了自己该干什么，且完整地掌握了淘内C端业务仓库下的开发方式。

- 具体执行阶段 – 解耦实现

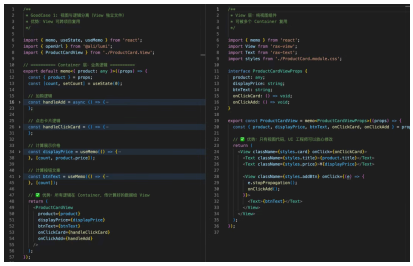
关于执行部分，逻辑内容前后端实现起来都大差不差，这部分内容方案确定以后AI产出的代码基本都能满足需求，这块重点讲下C端前端最关键的视觉部分。

C端 AI 编码不好用的一个主要原因就是C端逻辑与视觉耦合度太高，而 AI 又天生缺乏对视觉内容的感知力，此时如果一次性让AI把组件的逻辑代码和视觉都写完容易顾此失彼，导致问题直接上升了一个复杂度。

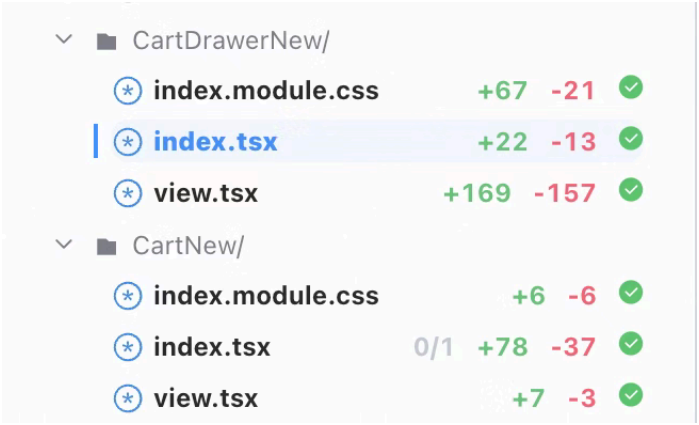
此时最理想的解法，还是尽可能的将视觉代码与逻辑代码分离，先让 AI 完成逻辑代码部分，再单独通过其他方案完成视觉组件的编写，再使用 AI 将逻辑与视觉组件进行绑定。基础的视图分离比较简单，就是首先假定一个抽象组件，包含了属性与事件，再和一个只负责绑定事件与属性的纯视觉组件结合即可。

Bad Case ❌	Good Case ✅
视图和逻辑耦合严重，每次对视图的修改都要同时影响到逻辑实现	视图和逻辑完全解耦，视图修改不再影响逻辑，业务层和视图层均可以快速迁移复用 （为了展示清晰所以采用两个文件的形式，实践中可以合并到一个组件，只要保留这个视图分离的设计思维即可）

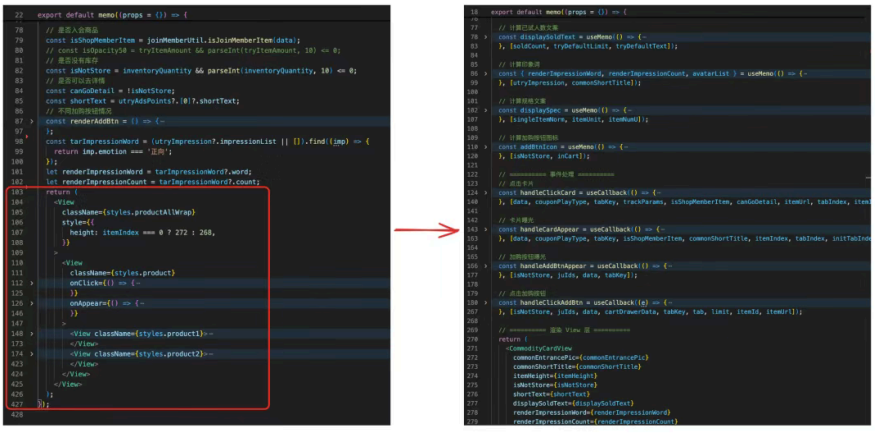
```tsx
export default memo<{ product: any }>((props) => {
  const { product } = props;
  const [count, setCount] = useState(0);

  // 加购逻辑
  const handleAdd = async () => {
    await fetch('/api/cart', { method: 'POST' });
    setCount(count + 1);
  };

  // ❌ 问题：视图和逻辑混在一起，改样式会碰到业务代码
  return (
    <View className={styles.card} onClick={() => {
      // 内联逻辑判断
      if (count > 3) {
        alert('已加购多次');
      }
      openUrl(product.url);
    }}>
      <Text className={styles.price}>
        {/* 价格计算逻辑混在 JSX 里 */}
        ¥{count > 2 ? (Number(product.price) * 0.9).toFixed(2) : product.price}
      </Text>
      <View className={styles.addBtn} onClick={(e) => {
        e.stopPropagation();
        if (count >= 5) {
          alert('已达上限');
        }
        handleAdd();
      }}>
        {/* 文案逻辑混在渲染里 */}
        <Text>{count === 0 ? '加购' : count < 3 ? `再购${5-count}` : `已购${count}件`}</Text>
      </View>
    </View>
  );
});
```



视图分离还有个好处就是极大地减少了前端的 CR 压力，比如一个视图分离后的购物车组件，CR 时只需要重点查看主逻辑 index.tsx 的代码变更，审查压力瞬间少掉大半。



重构过程中，也经常会遇到视图和逻辑绑定过深，无法复用 视觉/逻辑 代码的情况，这时候也可以直接让 AI 进行代码拆解，产出更加纯粹的 逻辑/视觉组件。比如这个需求中的商卡包含大量逻辑，我想实现新的卡片样式还得从原来 400 行的视觉组件里挑出来所有逻辑代码，简直没有天理。但是让AI将组件改造成视图分离的结构后，再通过D2C产出新的卡片组件，在进行状态与事件的绑定即可，后续迭代也会更加清晰。



基于以上思路，还可以进一步设计视图分离的组件库，预设组件的事件，由调用方进行视觉组件的实现，完成事件的绑定，做到最大化的逻辑复用。比如，我们业务有需要在不同场景中复用的feeds模块，为了保证最大化的逻辑复用，我们将 tab 渲染的部分交给调用方，调用方自己进行 tab 部分的视觉实现，只要给对应的元素做好事件绑定即可。

```
<Feeds
  getCardHeight={() => 216}
  dataSource={dataSource}
  TabRender={(props: TabProps) => {
    // tab在本方法中渲染，props会传入环境参数
    // 环境参数包括：
    // currentIndexL1: number; 当前选中的tabL1索引
    // onChangeL1: (index: number) => void; 切换tabL1回调
    // onChangeL2: (index: number) => void; 切换tabL2回调
    // dataSourceRef: { current: any[] }; 数据源引用
    // tab实现方式不限定，完成了onChangeL1和onChangeL2的绑定即可
    const { currentIndexL1, dataSourceRef, onChangeL1, onChangeL2 } = props;

    return (
      <>
        <Tab
          tabItems={dataSourceRef.current}
          tabRender={(index, isActive, item) => {
            return (
              <div onClick={() => onChangeL1(index)} >
                {item.titleL1}
              </div>
            );
          }}
        />
        {dataSourceRef.current.map((tab, index) => {
          return (
            <Tab
              tabRender={(index, isActive, item) => {
                return (
                  <div onClick={() => onChangeL2(index)}>
                    {item.titleL2}
                  </div>
                );
              }}
```

- 后期沉淀

完成需求后，可以重新梳理整个流程中的问题与可以复用的内容，进一步完成资产沉淀，这部分内容前期的生成和调整都会比较费劲，但是基本几个中型需求认真跑下来的沉淀，就可以覆盖很多日常开发的内容了，然后就可以逐步进入坐享其成的阶段。当日常开发场景枚举到80%以后，AI 会越来越像我们延伸出的双手，不是胡编乱造，而是把我们脑海中的代码搬运到它们应该存在的地方。

组件沉淀

基于已有的视图分离结构，业务逻辑组件的可复用度已经不再受限于视觉稿，而剥离了业务逻辑的视觉素材，也可以快速的应用到各个项目。

知识文档迭代

虽然在前置准备期已经提前进行了知识文档的生成，但是 AI 大概率还是会有理解偏离的情况，这时候就要对已有的文档进行补充说明。比如：我提供了如何创建迭代的文档后，发现 AI

还是会自由发挥，导致流程执行错误。于是我针对几类常见问题补充了严格约束，通过运行时的及时修正来保证文档的有效性。

```
# O2 迭代和变更创建流程

## 执行契约（自动化 / AI 必读，优先于一切「用户只说了半句话」）

以下条款：**不满足则视为本次任务失败**，禁止以「用户没提」「等确认」「怕绑错单」为理由提前结束对话。

### 禁止行为（违反即视为未读本文档）

1. **禁止** 只执行「创建迭代」就结束，除非用户明确说「只要空迭代不要变更」。
2. **禁止** 以「未提供 `aoneReqId`」为借口停机：必须先用协作 MCP **查询或搜索**（见下文「前置准备」），再创建变更；仍无法确定时，**选用与需求语义最接近的一条并注明假设**，不得空停。
3. **禁止** 把「本地切分支」只写成文档条文而不执行：只要工作区存在对应 Git 仓库，执行者 **必须自己运行** `git fetch` / `git checkout`。
4. **禁止** 用「用户指令字面很短」缩小范围：**本文档是范围的上位依据**，口语中的「创建迭代」按上表 MVP 理解。

---      You, 2周前 • skill创建 …

## 前置准备

1. 确认应用信息：从项目的 `abc.json` 获取 `group` 和 `project`，应用标识格式：`{group}/{project}`
2. 确认分支名称：若无约定，使用 `feat/` 或 `fix/` + 简短英文语义，与迭代名称对齐
3. 确认工作项 ID（`aoneReqId`），**必须通过 MCP 主动获取**，建议顺序（任一成功即可，**勿跳过**）：
   - **优先**：`coop::query_recent_workitem_list`，`stamp: "Req"`，筛选 `status` 为「开发中」「评审中」「已选择」等进行中状态，取与当前需求最相关的一条的 `id`
   - **补充**：`coop::search_workitem_title`，`subject` 填业务关键词，在相关结果中选一条进行中的 `id`

**禁止**因「怕关联错需求」而整段位不创建变更；若必须猜测，创建变更后在回复中写明 Aone 链接或 ID，便于人类纠正。

## 操作步骤

### 1. 创建迭代

调用 `o2::add_iteration` 创建迭代:
```

工作流沉淀

完成一次需求以后，最重要的就是review整个实现流程，识别有没有 可标准化/重复性强/涉及文件多 的可沉淀流程，比如这个需求就有多个可以落成简单 Skill 的工作项，标准工作流的沉淀可以让 AI 越来越可控。

- 后端 ald solution 创建，分步修改相应文件；
- 在前面的基础上，进行前后端流程串联，如：solution –> 接口文档 –> 前端调用函数生成；
- 前端天马配置项的新增与相应的读取代码生成；
- 逻辑代码生成 –> 调用D2C工具生成视觉组件 –> 进行逻辑与视图的绑定。

04 团队介绍

本文作者卓屿，来自淘天集团–天猫新品营销技术团队。我们致力通过大数据、人工智能打造领先的数字化新品营销平台，服务于天猫新品全链路增长，面向品牌商家构建从新品研发、新品孵化到新品上新的一体化解决方案，负责「天猫小黑盒」/「天猫U先」/「TMIC」（天猫新品创新中心）/「淘系新品运营平台」等淘系核心的新品与新客业务，帮助商家连接淘系站内外流量、营销资源与数据，做规模化新品经营与确定性增长。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.6万

留言

写留言

还没有留言，来写首评